Fleet Wise

Complete System Documentation & Failure Analysis

Prepared 20 July 2026 (rev. 5) · Live at fleet-wise-delta.vercel.app · Source: github.com/Dobgim/fleet-wise

1. What Fleet Wise Is

Fleet Wise is a web application that helps anyone who owns vehicles — from a private person with two cars to a company running a fleet of trucks — stop losing money to surprise breakdowns. Users record their vehicles and every service performed (oil change, brakes, tires, battery, engine work), and the system turns that history into:

- **Maintenance alerts** — which services are overdue or coming due, based on standard service intervals (oil ≈ 6 months, brakes/tires ≈ 12 months, battery ≈ 24 months).
- **Cost intelligence** — spend per vehicle, monthly cost trends, and detection of vehicles whose repair pattern is abnormal (e.g. three brake jobs in six months), which usually signals a deeper problem or a bad repair shop.
- **An AI copilot** — a chat where the user asks questions in plain language ("Which vehicle costs me the most?", "When is the next oil change for TRK-012?") and gets answers grounded in *their own* data.

The business model is subscription-based: a free tier with limited capacity, and paid tiers (Premium $20/month, Business $100/month) that raise or remove the limits.

2. System Architecture

```
|    Security      |   |    from the database   |
+------------------+   +------------------------+
```

Layer	Technology	Role
Frontend	Next.js 16, React 19, TypeScript, Tailwind CSS 4	All pages and UI. Mobile-first, ChatGPT-style navigation drawer, dark-mode aware.
Hosting	Vercel (Hobby tier)	Builds from the GitHub `main` branch automatically on every push; serves the site globally.
Authentication	Supabase Auth	Email + password signup with email confirmation. Sessions carried in cookies, refreshed by `proxy.ts`, which also redirects signed-out visitors away from app pages.
Database	Supabase Postgres	`0001_init.sql`: organizations, memberships, vehicles, service_records, subscriptions, ai_summaries, ai_usage — all protected by Row-Level Security. `0002_ai_metering.sql` adds tamper-proof quota functions.
AI	OpenAI `gpt-4o-mini` via `/api/copilot`	Server-side route; the API key never reaches the browser. Falls back to a built-in rules engine if the LLM is unreachable.
Email	Supabase built-in (SMTP upgrade pending)	Confirmation emails. Branded HTML template prepared; requires custom SMTP to activate.
Working data	Supabase Postgres	Vehicles, service records, plan and the monthly AI-usage counter all live in the cloud database, scoped by organization. Nothing about the fleet is kept in the browser.

3. The Multi-Tenant Data Model

Every row of business data belongs to an **organization** — an invisible workspace, not a legal company. A private owner's "garage" and a logistics firm's fleet are the same concept. On first sign-in the app creates the user's organization automatically (named from signup, or auto-named like "joshua's garage").

Row-Level Security (RLS) is the core safety mechanism: the database itself refuses to return or accept rows unless the signed-in user has a membership in that row's organization. Even if the application code had a bug — or someone called the database API directly with the public anon key — Postgres blocks cross-tenant access. Two helper functions (`is_org_member`, `is_org_owner`) implement the check; writes to billing and AI-metering tables are reserved for trusted, privileged database functions only.

4. The AI Copilot — How It Actually Works

The AI is **not trained** on the user's data, and no training is ever needed. It works by *grounded prompting*:

1. The user asks a question in the chat. The browser sends **only the question** to the app's own server route `/api/copilot`.
2. The server identifies the user from their session cookie, consumes one question against their plan (see §4.1), then reads that organization's vehicles and service history *from the database*.
3. It builds a text context (one line per vehicle and per service record), attaches a strict system prompt — *answer only from this data; never invent vehicles, dates, or costs; say plainly when data is missing* — and calls OpenAI's `gpt-4o-mini` model (fractions of a cent per question).
4. The answer returns to the chat labelled **"AI answer."** If the OpenAI call fails for any reason (no key, no credit, outage), the app computes an answer locally with its rules engine and labels it **"Offline answer (rules)."** The chat never breaks.

Because the context is rebuilt from the database on every question, the AI is always current — another reason training is unnecessary. Because the context comes from the database rather than the request body, a user cannot forge a payload to ask about vehicles that are not theirs.

4.0a Starter prompts come from the user's own fleet

The suggested questions under the chat are generated from the signed-in user's vehicles and records, never from fixed examples. A user with one Toyota sees "Does CE-4521 need servicing soon?"; a user with several sees fleet-wide comparisons. A brand-new user with no vehicles is guided to add one rather than shown prompts about cars that do not exist.

4.0b Photo scanning to fill the vehicle form

Adding a vehicle asks for details a non-technical owner may not know — the VIN especially. The "Scan a photo instead" button lets the user photograph the car, its number plate, the registration document or the odometer; a vision model (`gpt-4o-mini` with image input) reads it and pre-fills registration, make, model, VIN and mileage. The user always reviews and confirms before saving — the AI fills the form, it does not create the record. The image is downscaled in the browser before upload, and the same "never invent a value" rule applies: an unreadable field returns blank, never guessed. Vision costs more than a text question, so a scan draws about 3,000 tokens from the same daily budget and is refused if the budget cannot cover it.

4.1 Cost control: daily token budgets

Usage is metered in **tokens**, the unit the model provider actually bills, not in questions. This matters because a question is not a fixed cost: the entire fleet context travels with every request, so the same

question from a two-car owner and from a 200-truck company can differ in price by a factor of fifty. Counting questions would have let the largest customers cost the most while paying the least.

No model training is involved in the measurement. Every OpenAI response includes a `usage` block reporting exactly what that call consumed, and the server debits that real number.

Plan	Daily tokens	Roughly	Token cost to you per day (gpt-4o-mini)
Free	5,000	~4 questions	≈ $0.002
Premium	50,000	~40 questions	≈ $0.02
Business	100,000	~80 questions	≈ $0.03

The flow per request: the API asks Postgres whether the org has at least 600 tokens of headroom left today; if not it refuses with HTTP 402 and tells the user when the budget refills. If allowed, the reply length is capped to what remains, so a single answer cannot overshoot far. After the model responds, `record_ai_tokens()` debits the true amount. Failed calls — outage, no credit — cost the user nothing. Budgets are keyed by UTC calendar day, so they refill automatically at midnight UTC with no scheduled job required.

All three functions (`get_ai_budget` , `check_ai_budget` , `record_ai_tokens`) are SECURITY DEFINER and are the only writable path into the usage table, so the counter cannot be edited by a user.

5. Plans & Monetization Mechanics

Plan	Price	Vehicles	AI tokens / day
Free	$0	5	5,000
Premium	$20 / month	50	50,000
Business	$100 / month	Unlimited	100,000

Limits are enforced by the database, not the browser. Budgets refill every day at UTC midnight, so a user who runs out simply waits rather than being locked out for a month — friendlier to the user and gentler on your costs at the same time. **Payment collection is still not wired**: the Subscribe button switches plans instantly without charging, so the full journey can be tested. A payment provider is the last remaining step before the product can take money (see §9).

6. Current State: What Is Real vs. What Is Simulated

Component	Status
UI — all pages, mobile & desktop	✅ Live; ChatGPT-style mobile navigation (hamburger drawer, centered brand, full-screen chat)
Sign up / sign in / email confirmation	✅ Live (Supabase Auth), with plain-language error messages (rate limits, wrong password, unconfirmed email, etc.)
Vehicles & service data	✅ Stored in Supabase Postgres under Row-Level Security; synced across devices and covered by Supabase backups
Plan limits & token metering	✅ Enforced server-side in Postgres (`0005_token_budgets.sql`). Verified live: a free org capped at 5,000, a 4,600-token debit correctly refused the next request, and upgrading raised the ceiling to 100,000 immediately
Plan tampering protection	✅ `0003` applied; self-upgrade and usage-table writes both refused (HTTP 403) in live testing
AI chat with OpenAI + rules fallback	⚠️ Deployed and metered; LLM answers pending OpenAI billing credit — until then every answer is the rules-engine fallback
Chat interface	✅ Rebuilt in the familiar assistant style: no bordered container, plain-text replies, single composer pill with a brand-blue arrow button
Fleet-specific starter prompts	✅ Suggestions generated from the user's own vehicles and records; empty state guides new users to add a vehicle
Photo scanner for adding vehicles	⚠️ Built and deployed; pending OpenAI billing credit to read images. Endpoint auth and validation verified; vision quality untestable until credit is added
Brand styling	✅ Logo blue applied across the wordmark ("Fleet" + blue "Wise"), buttons, active nav and chat bubbles, light and dark
Email maintenance reminders	✅ Built and running in production. Daily job at 07:00 UTC, branded email, per-garage opt-out, duplicate-send protection. Delivery of a real reminder is still untested (no garage has had anything overdue yet)
Payments (Stripe / alternatives)	❌ Not built — subscribing is simulated
Branded signup email	⚠️ Template ready; email confirmation currently switched off for testing. Turn back on once SMTP is connected

7. Failure Analysis — How and Why This System Will Fail

This section is deliberately blunt. Each risk lists the mechanism of failure and its mitigation.

7.1 Data durability RESOLVED

Former mechanism: an earlier revision of this document reported fleet data as living in browser localStorage, where clearing browsing data or switching phones would destroy it. **That assessment was incorrect** — the cloud data layer shipped with the initial build. Vehicles, service records and plan are read from and written to Supabase Postgres, scoped per organization by RLS.
Residual risk: the app has no export function, so a user cannot take their history elsewhere, and there is no undo for the "Clear all data" button. Recovery depends entirely on Supabase's backup retention, which is limited on the free tier.
Recommended: add CSV export and a soft-delete (archive) instead of a hard delete.

7.2 Revenue-limit bypass RESOLVED

Former mechanism: the AI-question counter lived in localStorage and the API route trusted the caller, so a user could edit their browser storage — or call the endpoint directly with a forged fleet payload — and consume unlimited questions billed to the operator's OpenAI account.
Fix applied (`0002`, then `0003`, now `0005`): metering moved into Postgres, written only by privileged functions that derive the caller's organization from their session; the plan column was locked with column-level privileges so nobody can grant themselves a paid tier; and the API rejects unauthenticated calls (401), refuses exhausted budgets (402), and builds the AI context from the database rather than the request body, so a forged payload is ignored. All four attacks were re-tested against the live system on 20 July and refused.
Residual risk: no per-minute rate limit. A Business account cannot exceed 100,000 tokens a day, which caps the damage at roughly $0.03 — but a burst could still be noisy. Worth adding before launch.

7.3 OpenAI dependency & cost exposure MEDIUM

Mechanism: (a) The account currently has no billing credit — every request returns "insufficient_quota", so users silently receive rules-engine answers instead of AI ones. (b) If credit runs out unnoticed mid-month, answer quality degrades without alerting anyone. (c) The API key was shared in plain text during development and should be treated as semi-exposed.

Fix: add billing credit with a hard monthly spend cap in the OpenAI dashboard; rotate the key and set a usage alert; log how often the fallback path is taken.

7.4 Large fleets will strain the AI context MEDIUM

Mechanism: every service record for the organization is sent with every question (capped at 300 vehicles / 2,000 records). A Business-tier customer with years of history will eventually exceed the model's context window or make each question needlessly expensive — a design that works for 10 vehicles fails at 1,000.
Fix: query only the rows relevant to the question, summarize old history, and cache monthly summaries in `ai_summaries` (the table already exists).

7.5 Free-tier infrastructure limits MEDIUM

Mechanism: Supabase free projects are **paused after about a week of inactivity** — sign-ins then fail until the project is manually restored. Vercel's Hobby tier has bandwidth and function limits and is not licensed for commercial use at scale. Supabase's built-in email service allows only a few messages per hour, which already caused failed signups during testing.
Fix: upgrade Supabase and Vercel to paid tiers before real users arrive; move email to a custom domain with a proper provider.

7.6 AI answer quality LOW

Mechanism: the system prompt forbids invention, but language models can still miscalculate (for example summing costs across many records) or sound confident when uncertain. Maintenance predictions use fixed intervals rather than manufacturer schedules or actual mileage accumulation — a taxi and a weekend car both get "oil every 6 months".
Fix: compute numeric aggregates in code and hand them to the model pre-calculated; add usage-based intervals per vehicle; keep the "AI answer" label so users always know the source.

7.7 Operational blind spots MEDIUM

Mechanism: there is no error monitoring, no uptime alerting, and no automated test suite. Failures will be discovered by users rather than by the operator. The project also depends on a single developer machine, which already hit a full-disk failure during this build.

Fix: add Sentry (free tier) for error reporting, an uptime ping against the live URL, and a minimal test suite covering quota logic and the insights engine before adding more features.

7.8 Trust & brand risks LOW

Mechanism: confirmation emails currently arrive from "Supabase Auth", which reads as phishing to cautious users; the app lives on a `vercel.app` subdomain rather than its own domain; and there are no Terms of Service or privacy policy despite storing user emails and, soon, payment status.
Fix: a custom domain, a custom email sender, and basic legal pages before charging money.

8. Priority Roadmap

#	Action	Addresses
1	~~Cloud data layer~~ — **done** (shipped with the initial build)	§7.1
2	~~Server-side plan & quota enforcement~~ — **done** (`0002_ai_metering.sql`); per-minute rate limiting still open	§7.2
3	**Payment checkout + webhooks writing to the `subscriptions` table** ← next	Simulated billing
4	OpenAI credit + spend cap + key rotation	§7.3
5	Custom domain, proper email sender, branded emails, legal pages	§7.5, §7.8
6	~~Scheduled daily reminder emails~~ — **done**; relevant-rows AI context still open	§7.4
7	Error monitoring, uptime checks, tests	§7.7
8	Per-minute rate limiting on the AI endpoint	§7.2 residual

9. Monetization Playbook — How Fleet Wise Makes Money

9.1 Core engine: freemium subscriptions

The product implements the classic AI-SaaS funnel. The free tier is a *demonstration machine*: a user adds up to 5 vehicles, gets real alerts, and asks the AI 10 questions. The moment the product proves useful, the user meets a limit at exactly the point of highest motivation — mid-conversation with the copilot, or adding a 6th vehicle — and the upgrade prompt appears right there. Two paid exits: **Premium $20/mo** (individuals, small garages) and **Business $100/mo** (companies).

9.2 Unit economics — why the margins work

	Free user	**Premium $20**	**Business $100**
Daily token ceiling	5,000	50,000	100,000
Worst case AI cost per month, budget maxed every single day	≈ $0.05	≈ $0.50	≈ $0.90
Realistic cost (users rarely max out)	< $0.01	≈ $0.10	≈ $0.25
Infrastructure share (paid tiers divided by users)	< $0.10	< $0.50	< $1.00
Gross margin	— (marketing cost)	**≈ 97%**	**≈ 98%**

The decisive property of daily token caps is that **the worst case is now known and small**. Even if every Business customer exhausted 100,000 tokens every day of the month, the AI bill per customer is under a dollar against $100 of revenue. There is no scenario — short of a bug — in which usage costs outrun subscription income. The real costs are fixed (hosting tiers, domain, email service — roughly $50–75/month at small scale), so **three to four Premium subscribers cover the entire infrastructure**; everything beyond that is margin.

9.3 Getting paid — the practical part

- **Global cards:** Stripe is the default, but it does not onboard merchants in every country. Where Stripe is unavailable, use a **merchant of record** such as Paddle or Lemon Squeezy — they sell on your behalf worldwide, handle VAT and sales tax, and pay out internationally. Slightly higher fees (around 5%), far less compliance work.
- **African markets:** if targeting users in Cameroon or the wider region, card penetration is low — integrate **Mobile Money** (MTN MoMo, Orange Money) through an aggregator such as Flutterwave or

CinetPay, and price locally. A flat $20 price point is mispriced for that market.

- **Annual billing:** offer two months free for paying yearly ($200/yr, $1,000/yr). Annual prepayment funds growth and sharply reduces churn.

9.4 Growth levers beyond subscriptions

Lever	How it works	When
Per-vehicle enterprise pricing	Fleets of 100+ vehicles pay per vehicle (e.g. $1.50/vehicle/month) instead of a flat tier, so revenue scales with customer size.	After the first Business customers
AI token top-ups	A user who exhausts today's budget buys another 50,000 tokens for $3 instead of waiting for midnight. The daily cap creates this moment naturally, several times a month. Near-100% margin.	Easy add-on once checkout exists
Garage / mechanic partnerships	When the AI says "brakes due", show a booking link to a partner garage and charge the garage a referral fee per booked job.	Needs a user base first
Fleet reports as a paid artifact	A monthly branded PDF cost report per fleet — the same generation pipeline that produced this document. Sell as a Business perk or standalone.	Quick win
White-label	Insurance companies or leasing firms rebrand Fleet Wise for their clients under a flat monthly licence.	Long term

9.5 The one remaining prerequisite

Two of the three prerequisites are now met. Data lives in the cloud database (§7.1) and plan limits are enforced server-side, so paying is no longer optional (§7.2). **The only remaining blocker to revenue is a payment provider** wired to the existing `subscriptions` table — roadmap item 3. Once checkout is connected and the OpenAI account is funded, the product can charge customers.

End of document — Fleet Wise, 20 July 2026 (rev. 3).